ΟΝΟΜΑΤΕΠΩΝΥΜΟ Ναταλία Ρούσκα
ΑΜ 1092581
ΕΤΟΣ ΦΟΙΤΗΣΗΣ 2022

Ανάπτυξη Φόρμας για την Εισαγωγή Νέων Χρηστών σε Ιστοσελίδα με τη Χρήση του Framework Flask

Περίληψη

Στην παρούσα εργασία χρησιμοποιούμε το framework Flask της Python για την ανάπτυξη μια ιστοσελίδας η οποία περιέχει μια φόρμα που επιτρέπει εγγραφές νέων χρηστών. Οι εγγραφές αποθηκεύονται σε αρχείο κειμένου. Στην εισαγωγή παρουσιάζονται εν συντομία όλες οι τεχνολογίες που χρησιμοποιήθηκαν. Στη συνέχεια παρατίθενται τα βήματα δημιουργίας του περιβάλλοντος στο οποίο τρέχει η εφαρμογή, περιγράφεται η λειτουργία της εφαρμογής και δίνονται παραδείγματα χρήσης του προγράμματος. Τέλος, παρουσιάζεται ο τρόπος φιλοξενίας της εφαρμογής σε δημόσια διεύθυνση μέσω της υπηρεσίας Heroku.

I. ΕΙΣΑΓΩΓΗ

Στην παρούσα παράγραφο δίνεται μια σύντομη περιγραφή των τεχνολογιών που χρησιμοποιούνται στο πλαίσιο της εργασίας [1]. Ο παγκόσμιος ιστός (worldwide web) αποτελεί ίσως τη σημαντικότερη εφαρμογή του διαδικτύου και επιτρέπει την επικοινωνία φυλλομετρητών (browsers) με εξυπηρετητές (web servers) με τη βοήθεια του πρωτοκόλλου HTTP (hyper-text transfer protocol). Το πρωτόκολλο HTTP χρησιμοποιεί δύο βασικούς τύπους μηνυμάτων (α) αιτήματα GET ή POST από τον φυλλομετρητή προς τον εξυπηρετητή και (β) αποκρίσεις του εξυπηρετητή προς τον φυλλομετρητή.

Τα αρχεία που διαβάζουν οι φυλλομετρητές και αποτυπώνουν το περιεχόμενό τους στην οθόνη του υπολογιστή είναι γραμμένα στη γλώσσα επισημείωσης περιεχομένου (markup) HTML. Με την HTML ορίζεται το περιεχόμενο και η δομή μιας ιστοσελίδας, οι κανόνες μορφοποίησης CSS ορίζουν την εμφάνιση των στοιχείων HTML της ιστοσελίδας, ενώ με τη γλώσσα Jinja2 είναι εφικτός ο σχεδιασμός ιστοσελίδων με παραμετρικό τρόπο.

Η βιβλιοθήκη Flask προσφέρει τη δυνατότητα δρομολόγησης και εξυπηρέτησης μηνυμάτων που φτάνουν web server προς μια εφαρμογή εξυπηρετητή γραμμένης στη γλώσσα Python. Ο web server μπορεί να τρέχει τοπικά στην διεύθυνση:θύρα 127.0.01:5000. Μέσω όμως της υπηρεσίας Heroku (Platform as a Service, PaaS) που χρησιμοποιεί χρησιμοποιεί υπολογιστικές υπηρεσίες της AWS (Amazon Web Services), υπάρχει η δυνατότητα φιλοξενίας της εφαρμογής Flask σε δημόσια διεύθυνση χωρίς όμως δυνατότητα αποθήκευσης των εγγραφών σε αρχείο.

Η παρούσα εργασία αφορά την ανάπτυξη μια εφαρμογής που εγγράφει μέσω μια φόρμας νέους χρήστες και αποθηκεύει τα στοιχεία εγγραφής σε ένα αρχείο κειμένου χρησιμοποιώντας όλες τις παραπάνω προαναφερθείσες τεχνολογίες διαδικτύου.

Στην Παράγραφο ΙΙ περιγράφονται αναλυτικά όλα τα βήματα για την εγκατάσταση του περιβάλλοντος, η λειτουργία της εφαρμογής, παραδείγματα χρήσης και η φιλοξενία στην υπηρεσία Heroku. Τέλος, στην Παράγραφο ΙΙΙ γίνεται μια σύνοψη με τα συμπεράσματα της εργασίας.

II. ΑΝΑΠΤΥΞΗ ΘΕΜΑΤΟΣ

Α. Αρχεία Κώδικα που αναπτύχθηκαν

Για την υλοποίηση της εφαρμογής αναπτύχθηκαν 4 αρχεία κώδικα, `base.html`, `signup.html`, `signuperror.html`, `natserver.py`.

Η `base.html` περιέχει τη βασική μορφοποίηση που κληρονομούν οι `signup.html`, `signuperror.html`.

Η `signup.html` είναι η βασική ιστοσελίδα που επιστρέφει ο εξυπηρετητής στα αιτήματα GET που λαμβάνει και περιέχει τη φόρμα εγγραφής, όπου χρήστης καλείται να εισάγει τα στοιχεία του.

Η `signuperror.html` είναι η ιστοσελίδα που επιστρέφει ο εξυπηρετητής σε περίπτωση εισαγωγής ήδη καταχωρημένου ονόματος. Το περιεχόμενό της είναι βασικά ίδιο με την `signup.html`, με τη διαφορά ότι τυπώνει ένα μήνυμα ότι το όνομα είναι ήδη καταχωρημένο.

Το `natserver.py` υλοποιεί τον εξυπηρετητή που δέχεται τα αιτήματα GET και POST από τον φυλλομετρητή και ανταποκρίνεται αντίστοιχα.

1. Αν δεχθεί μήνυμα GET επιστρέφει την ιστοσελίδα `signup.html`.
2. Αν δεχθεί μήνυμα POST, διαβάζει πρώτα τα στοιχεία που εισήγαγε ο χρήστης στη φόρμα και πρώτα ανοίγει το αρχείο `users.txt` και ελέγχει αν το όνομα του χρήστη είναι ήδη καταχωρημένο.
   a. Αν το όνομα υπάρχει ήδη, επιστρέφει την ιστοσελίδα `signuperror.html` στον φυλλομετρητή η οποία περιέχει ένα μήνυμα προς τον χρήστη.
   b. Αν το όνομα δεν υπάρχει, ανοίγει πάλι το αρχείο `users.txt` και προσθέτει στο τέλος του αρχείου τα στοιχεία με `name`, `email`, `password`, `city`, που έγραψε ο χρήστης στη φόρμα, καθώς και `date`, και `hour` που έγινε η εισαγωγή. Μετά επιστρέφει την ιστοσελίδα `signup.html` στον φυλλομετρητή.

Β. Εγκατάσταση Περιβάλλοντος

Η εργασία αναπτύχθηκε σε φορητό υπολογιστή DELL 64bit που τρέχει WINDOWS 10.

Στο φάκελο `myproject-local` τοποθετούνται όλα τα απαραίτητα αρχεία για να τρέξει η εφαρμογή.

Δημιουργούμε μέσω του terminal του vs code ένα εικονικό περιβάλλον στο οποίο εγκαθιστούμε το Flask με την εντολή

```
pip install Flask
```

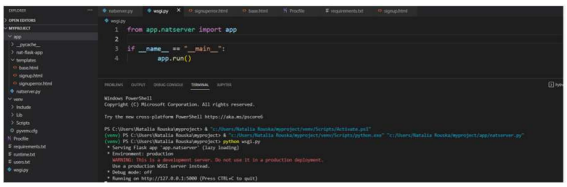
Όλες οι απαραίτητες βιβλιοθήκες βρίσκονται τώρα στον φάκελο `venv`.

Δημιουργούμε τα python αρχεία `wsgi.py` και `natserver.py` και τρέχουμε την εφαρμογή μας με την εντολή

```
python wsgi.py
```

Μαζί με την βιβλιοθήκη Flask έχουμε κατεβάσει ένα web server που τρέχει τοπικά στον υπολογιστή μας στην διεύθυνση:θύρα 127.0.01:5000. [2]

Το περιβάλλον ανάπτυξης της εφαρμογής παρουσιάζεται στο παρακάτω Σχ. 1.



Σχήμα 1. Περιβάλλον ανάπτυξης εφαρμογής.

Γ. Λειτουργία Εφαρμογής

Σε έναν φυλλομετρητή ο χρήστης μπορεί να αναγράψει την διεύθυνση:θύρα http://127.0.0.1:5000 και θα λάβει την παρακάτω εικόνα της ιστοσελίδας που φαίνεται στο Σχ. 2.

Σχήμα 2. Ιστοσελίδα εφαρμογής τοπικά.

Στο φάκελο `templates` βρίσκεται η ιστοσελίδα `signup.html` που καλείται με την μέθοδο `render_template()` της Flask.

Με το πάτημα του πλήκτρου Submit τα στοιχεία της φόρμας στέλνονται με την μέθοδο POST στην ίδια διεύθυνση `(action ="#" )` .

Τα στοιχεία που εισάγει ο χρήστης αποθηκεύονται σε αρχείο κειμένου με το όνομα users.txt. Στο ίδιο αρχείο καταχωρούνται επίσης η ημερομηνία και ώρα εισαγωγής των στοιχείων του χρήστη, όπως φαίνεται στο Σχ. 3.

```
users.txt
1
2  natalia  natalia.rouska@gmail.com 123456 Patras 12/06/2022 00:59:30
3  example name  example@gmail.com avfdddg Athens 12/06/2022 01:00:02
```

Σχήμα 3. Περιεχόμενα αρχείου `users.txt`.

Σε περίπτωση που ο χρήστης πληκτρολογήσει ένα όνομα που έχει ήδη καταχωρηθεί, η εφαρμογή δεν επιτρέπει την καταχώρηση και επιστρέφει ενημερωτικό μήνυμα και ο χρήστης καλείται να καταχωρίσει καινούρια στοιχεία.

Δ. Φιλοξενία Εφαρμογής στο Heroku

Στο φάκελο `myproject` βρίσκονται όλα τα απαραίτητα αρχεία για να τρέξει η εφαρμογή που πλέον εκτελείται από το URL https://nat-flask-app.herokuapp.com/ .

Παρακάτω περιγράφεται συνοπτικά η διαδικασία φιλοξενίας της εφαρμογής στο Heroku [3].

Εγκαθιστούμε τη βιβλιοθήκη `gunicorn`.

Δημιουργούμε ένα αρχείο `requirements.txt` με τα περιεχόμενα όπως φαίνονται στο παρακάτω Σχ. 4.

```
requirements.txt
1  click==8.1.3
2  colorama==0.4.4
3  Flask==2.1.2
4  gunicorn==20.1.0
5  itsdangerous==2.1.2
6  Jinja2==3.1.2
7  MarkupSafe==2.1.1
8  Werkzeug==2.1.2
9
```

Σχήμα 4. Περιεχόμενα `requirements.txt`.

Δημιουργούμε ένα αρχείο `Procfile` με το παρακάτω περιεχόμενο (Σχ. 5).

```
Procfile
1  web: gunicorn wsgi:app
```

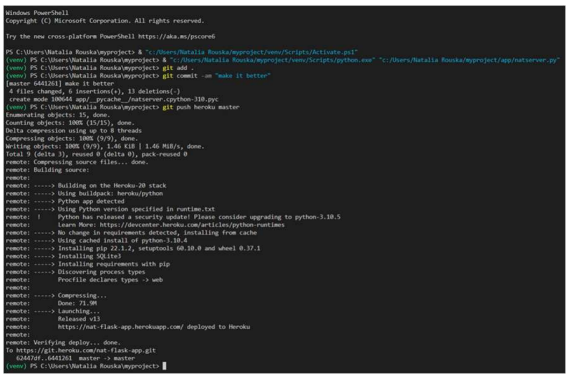
Σχήμα 5. Περιεχόμενο `Procfile`.

Δημιουργούμε ένα λογαριασμό Heroku.

Πραγματοποιούμε λήψη και εγκατάσταση του Heroku CLI. [4]

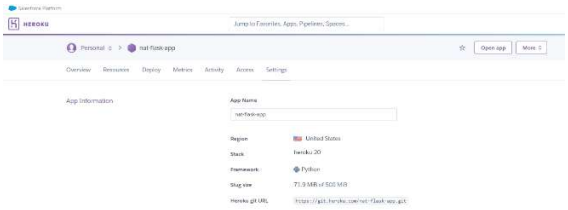
Στο terminal αυτό εκτελούμε τις εντολές όπως φαίνονται και στο Σχ. 6

```
git init
git add .
git commit -m "Initial Commit"
heroku create nat-flask-app
git push heroku master
```

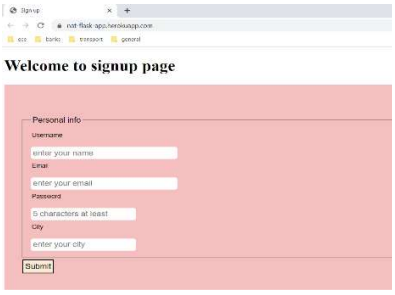


Σχήμα 6. Εγκατάσταση εφαρμογής στο Heroku.

Στο Σχ. 7 φαίνεται το Dashboard του Heroku και ο σύνδεσμος όπου φιλοξενείται η εφαρμογή, ενώ στο Σχ. 8 εμφανίζεται η ιστοσελίδα όπως διαβάζεται στον φυλλομετρητή στη δημόσια διεύθυνση https://nat-flask-app.herokuapp.com/ .



Σχήμα 7. Dashboard Heroku.



Σχήμα 8. Ιστοσελίδα εφαρμογής στο Heroku.

Η εφαρμογή που φιλοξενείται στο Heroku έχει την ίδια συμπεριφορά με αυτήν που τρέχει τοπικά. Δηλαδή, δεν επιτρέπει την εισαγωγή χρήστη με όνομα που έχει ήδη καταχωρηθεί. Παρόλο που δεν ανανεώνεται το αρχείο users.txt, η εφαρμογή αποδέχεται τα νέα στοιχεία που εισάγονται στην φόρμα. Μάλιστα, ο χρήστης καλείται να καταχωρήσει καινούριο όνομα όταν το προηγούμενο υπάρχει ήδη.

III. ΣΥΝΟΨΗ

Στην παρούσα εργασία υλοποιήθηκε μια διαδικτυακή εφαρμογή που εγγράφει μέσω μια φόρμας νέους χρήστες και αποθηκεύει τα στοιχεία εγγραφής σε ένα αρχείο κειμένου χρησιμοποιώντας το framework Flask. Η εφαρμογή ελέγχει αν υπάρχει ήδη καταχωρημένο το όνομα του χρήστη και επιτρέπει την εγγραφή μόνο νέων χρηστών. Αναπτύχθηκαν τα αρχεία των ιστοσελίδων .html που υλοποιούν τη φόρμα στην μεριά του φυλλομετρητή και τα αρχεία του εξυπηρετητή .py που υλοποιούν τον έλεγχο και την καταχώρηση των στοιχείων των χρηστών σε ένα αρχείο κειμένου. Η εφαρμογή τρέχει τόσο τοπικά, όσο και διαδικτυακά σε δημόσια διεύθυνση με τη χρήση της υπηρεσίας cloud Heroku. Δυστυχώς, στη δεύτερη περίπτωση δεν είναι δυνατή η επισκόπηση των αλλαγών στο αρχείο με τα στοιχεία των χρηστών, παρότι από τις δοκιμές που έγιναν φαίνεται ότι καταχωρούνται σωστά γιατί σε δεύτερη προσπάθεια με το ίδιο όνομα καταχώρισης επιστρέφεται το μήνυμα λάθους.

ΒΙΒΛΙΟΓΡΑΦΙΑ

[1]._https://github.com/navouris/Y210-web-technologies/blob/main/handouts/Y210-Avouris-web-technologies-notes.md.

[2]._https://flask.palletsprojects.com/en/1.1.x/quickstart/#a-minimal-application.

[3]._https://www.geeksforgeeks.org/deploy-python-flask-app-on-heroku/.

[4]._https://devcenter.heroku.com/articles/getting-started-with-python?singlepage=true.

Βοηθητικό υλικό για την κατανόηση της λειτουργίας του Flask και παραδείγματα ανάπτυξης φόρμας

[5]._https://mathesis.cup.gr/courses/course-v1:ComputerScience+CS3.4+21F/course/

[6]._https://mathesis.cup.gr/courses/course-v1:ComputerScience+CS3.2+21A/course/

IV. ΠΑΡΑΡΤΗΜΑ

Παρακάτω παρατίθενται τα αρχεία με τον κώδικα που αναπτύχθηκε στα πλαίσια της εργασίας.

Αρχείο `base.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible"
content="IE=edge">
    <meta name="viewport" content="width=device-
width, initial-scale=1.0">
    <title> {% block title %}{% endblock title %}
</title>
    <style>
        input, button, legend{
            margin: 0.2em;
            padding: 0.2em;
            font-size: 1em;
            font-family:Arial, Helvetica, sans-
serif;
        }
        form{
            padding: 2em;
            background-color: #F4BFBF;
        }
        label{
            margin: 0.2em;
            padding: 0.2em;
            font-size: 0.8em;
            font-family:Arial, Helvetica, sans-
serif;
        }
        input{
            margin-top: 1em;
            border: 1px solid rgb(250, 219, 215);
            border-radius: 5px;
            box-sizing: border-box;
        }
        fieldset{
            margin-top: 2em;
        }
        button{
            background-color: antiquewhite;
        }

    </style>
</head>
<body>
    {% block body %}
    {% endblock body %}
</body>
</html>
```

Αρχείο `signup.html`

```
{% extends "base.html" %}
{% block title %} Sign up {% endblock title %}
{% block body %}
    <h1> Welcome to signup page</h1>
    <form action='#', method='POST'>
        <fieldset>
            <legend>Personal info</legend>
            <label for="name">Username</label><br>

            <input type="text" name="name"
placeholder="enter your name" size="30"
required><br>

            <label for="email">Email</label><br>
```

```
            <input type="text" name="email"
placeholder="enter your email" size="30"
required><br>

            <label
for="password">Password</label><br>
            <input type="password" name="password"
placeholder="5 characters at least" required><br>

            <label for="cities">City</label><br>
            <input list="cities" name="cities"
placeholder="enter your city" required>
            <datalist id="cities">
                    <option
value="Patras">Patras</option>
                    <option
value="Athens">Athens</option>
                </datalist>

            </div>


        </fieldset>
        <button>Submit</button>


    </form>
{% endblock body %}
```

Αρχείο signuperror.html

```
{% extends "base.html" %}
{% block title %} Sign up {% endblock title %}
{% block body %}
    <h1> Please choose a different name, that one
is already in use</h1>

    <form action='#', method='POST'>
        <fieldset>
            <legend>Personal info</legend>
            <label for="name">Username</label><br>

            <input      type="text"      name="name"
placeholder="enter     your     name"     size="30"
required><br>

            <label for="email">Email</label><br>
            <input     type="text"     name="email"
placeholder="enter     your     email"     size="30"
required><br>

            <label
for="password">Password</label><br>
            <input  type="password"  name="password"
placeholder="5 characters at least" required><br>

            <label for="cities">City</label><br>
            <input    list="cities"    name="cities"
placeholder="enter your city" required>
            <datalist id="cities">
                    <option
value="Patras">Patras</option>
                    <option
value="Athens">Athens</option>
                </datalist>

            </div>


        </fieldset>
        <button>Submit</button>


    </form>

{% endblock body %}
```

Αρχείο natserver.py

```
from datetime import datetime
import os
from flask import Flask, flash, render_template,
request, redirect
app = Flask(__name__)

@app.route('/',methods=["GET","POST"])
def signup():
    if request.method == 'POST':
        name=request.form['name']
        email=request.form["email"]
        password=request.form['password']
        city=request.form['cities']
        now = datetime.now()
        dt= now.strftime("%d/%m/%Y %H:%M:%S")
        if (name) in open('users.txt').read():
            print(name)
            return
render_template("signuperror.html")
        else:
            with open('users.txt','a') as file:
                file.write(f"\n{name}       {email}
{password} {city} {dt}")
                file.close()
            return render_template("signup.html")
    else:
        return render_template("signup.html")
```

Αρχείο wsgi.py

```
from app.natserver import app

if __name__ == "__main__":
        app.run()
```